HW5 B11103241 陳俊言 JamesChen

1. **Create a package including two publishers and two subscribers separated in four nodes. One publisher would send a message with a random integer from 0 to 255 to one subscriber 5 times in one second (5 Hz). The other publisher would send the room temperature detected with the temperature sensor to the other subscriber every 5 seconds. Build and run four nodes on a local computer.**

   完整 Code: clickme

   Video: clickme1

```python
import random
import rclpy
from rclpy.node import Node
from std_msgs.msg import UInt8
from rclpy.logging import LoggingSeverity
class RandomPublisher(Node):
#創建一個子類別RandomPublisher，負責處理發送隨機數字這個任務。他已經繼承父類別Node這個基底，Node這個

    def __init__(self):
    #每當做Node=RandomPublisher()，這個物件會被建立好(看不到)，接著自動呼叫__init__(把節點所需東西

        super().__init__('random_publisher')
        #super會去呼叫父類別，而這句super().__init__('random_publisher')則是呼叫父類別Node的initi

        self.pub = self.create_publisher(UInt8, 'random_byte', 10)
        #向ROS2登記一個Publisher物件，並存成(self.pub)這個屬性，這個屬性名稱可以自己取。後面的self.
        #( )內需分別是:訊息類別，topic名稱(供訂閱，要跟subscriber一樣)，佇列深度

        self.timer = self.create_timer(0.2, self.timer_cb)
        #建立一個timer物件，並存成self.timer這個屬性。每0.2秒執行一次回呼，self.timer_cb是回呼函式的
        #( )內需分別是，計數秒數second，需回呼函式

        self.get_logger().info('random_publisher started @5Hz on topic /random_byte')
        # self.get_logger()是Node提供的固定API，.info則是他的等級，取得這個節點專屬的logger，()裡面
        self.get_logger().set_level(LoggingSeverity.INFO)

    def timer_cb(self):
    #回呼函式。會被上面寫的timer呼叫
        msg = UInt8()
        #建立一個訊息物件
        msg.data = random.randint(0, 255)
        self.pub.publish(msg)
        self.get_logger().info(f'Published: {msg.data}')

def main():
    rclpy.init()
    node = RandomPublisher()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

```python
import rclpy
from rclpy.node import Node
from std_msgs.msg import UInt8

class RandomSubscriber(Node):
    def __init__(self):
        super().__init__('random_subscriber')
        self.sub = self.create_subscription(UInt8, 'random_byte', self.cb, 10)
        self.get_logger().info('listening: /random_byte')

    def cb(self, msg: UInt8):
        self.get_logger().info(f'random_byte = {msg.data}')

def main():
    rclpy.init()
    node = RandomSubscriber()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2. **Based on the codes in Exercise 1, write two publishers in one node and two subscribers in the other node in a package, which means there are only two nodes in this package instead of four nodes. Build and run two nodes on the same computer or on different computers.**

   完整 Code: clickme

   Video: clickme

```python
# hw51/hw51/multi_pub.py
import random, time, serial
import rclpy
from rclpy.node import Node
from rclpy.logging import LoggingSeverity
from std_msgs.msg import UInt8
from sensor_msgs.msg import Temperature

PORT = '/dev/ttyACM0'
BAUD = 9600

class MultiPublisher(Node):
    def __init__(self):
        super().__init__('multi_publisher')
        self.get_logger().set_level(LoggingSeverity.INFO)

        # Pub1: 隨機數
        self.rand_pub = self.create_publisher(UInt8, 'random_byte', 10)
        self.rand_timer = self.create_timer(0.2, self.rand_cb)  # 5 Hz

        # Pub2: 溫度
        self.temp_pub = self.create_publisher(Temperature, 'room_temperature', 10)
        self.temp_timer = self.create_timer(5.0, self.temp_cb)  # 每 5 秒

        # 串列連線
        self.ser = None
        try:
            self.ser = serial.Serial(PORT, BAUD, timeout=0.3)
            time.sleep(2.0)
            self.get_logger().info(f'Connected serial: {PORT} @ {BAUD}')
        except Exception as e:
            self.get_logger().error(f'Serial open failed: {e}')
```

```python
# hw51/hw51/multi_sub.py
import rclpy
from rclpy.node import Node
from rclpy.logging import LoggingSeverity
from std_msgs.msg import UInt8
from sensor_msgs.msg import Temperature

class MultiSubscriber(Node):
    def __init__(self):
        super().__init__('multi_subscriber')
        self.get_logger().set_level(LoggingSeverity.INFO)
        self.sub_rand = self.create_subscription(UInt8, 'random_byte', self.rand_cb, 10)
        self.sub_temp = self.create_subscription(Temperature, 'room_temperature', self.temp_cb, 10)
        self.get_logger().info('listening: /random_byte, /room_temperature')

    def rand_cb(self, msg: UInt8):
        self.get_logger().info(f'random_byte = {msg.data}')

    def temp_cb(self, msg: Temperature):
        self.get_logger().info(f'[{msg.header.frame_id}] T = {msg.temperature:.2f} °C (var={msg.variance:.2f})')

def main():
    rclpy.init()
    node = MultiSubscriber()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

3. **Build a system with ROS2 to remotely control at least three LEDs to turn on each LED in order every second. For example, after receiving a message, turn on the first LED. After receiving another message one second later, turn off the first LED and turn on the second LED. Repeat the pattern to control all LEDs. Publishers and subscribers should be run on different computers (e.g. one is on Raspberry Pi and one is on your computer).**

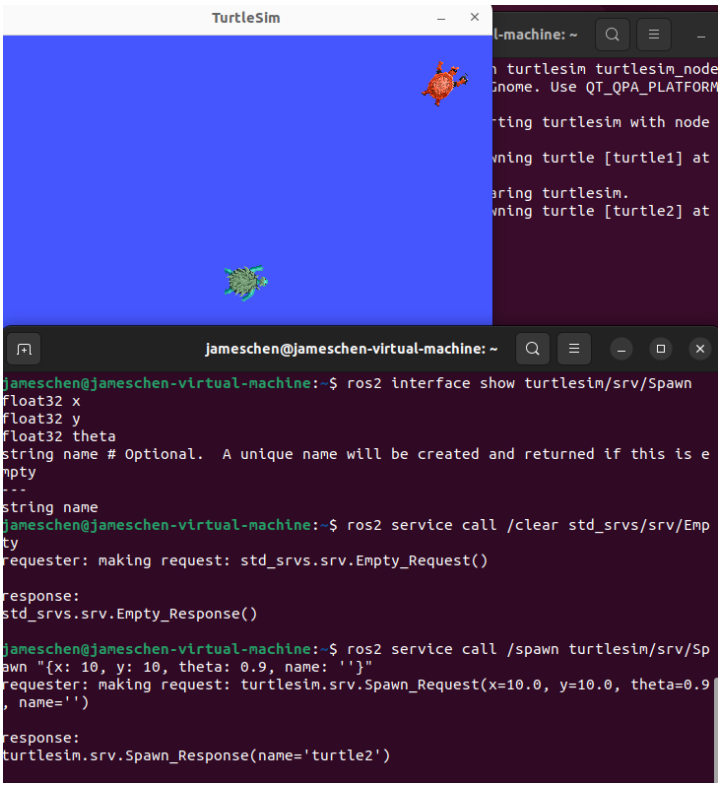
完整 Code: clickme

Video: clickme

```python
import rclpy
from rclpy.node import Node
from rclpy.logging import LoggingSeverity
from std_msgs.msg import UInt8

class LedCyclePublisher(Node):
    def __init__(self):
        super().__init__('led_cycle_publisher')
        self.get_logger().set_level(LoggingSeverity.INFO)
        self.pub = self.create_publisher(UInt8, 'led_index', 10)
        self.i = 0
        self.timer = self.create_timer(1.0, self.tick)  # 每秒送一次

    def tick(self):
        msg = UInt8()
        msg.data = self.i % 3          # 0,1,2,0,1,2...
        self.pub.publish(msg)
        self.get_logger().info(f'publish led_index = {msg.data}')
        self.i += 1

def main():
    rclpy.init()
    node = LedCyclePublisher()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node(); rclpy.shutdown()

if __name__ == '__main__':
    main()
```

```python
import rclpy
from rclpy.node import Node
from rclpy.logging import LoggingSeverity
from std_msgs.msg import UInt8
import serial, time

PORT = '/dev/ttyACM0'
BAUD = 9600

class LedSerialBridge(Node):
    def __init__(self):
        super().__init__('led_serial_bridge')
        self.get_logger().set_level(LoggingSeverity.INFO)
        self.sub = self.create_subscription(UInt8, 'led_index', self.on_index, 10)

        self.ser = None
        try:
            self.ser = serial.Serial(PORT, BAUD, timeout=0.2)
            time.sleep(2.0)
            self.get_logger().info(f'Connected serial: {PORT} @ {BAUD}')
        except Exception as e:
            self.get_logger().error(f'Serial open failed: {e}')

    def on_index(self, msg: UInt8):
        if not self.ser:
            self.get_logger().warning('Serial not available'); return
        idx = int(msg.data) % 3
        payload = f'{idx}\n'.encode('ascii')
        try:
            self.ser.write(payload); self.ser.flush()
            self.get_logger().info(f'sent to Arduino: {payload!r}')
        except Exception as e:
            self.get_logger().warning(f'Serial write error: {e}')

    def destroy_node(self):
        try:
            if self.ser: self.ser.close()
        finally:
            return super().destroy_node()
```

4. **In the ROS2 Humble document (https://docs.ros.org/en/humble/index.html), go through Tutorials / Beginner: CLI tools / Understanding services. Use ros2 service list -t and ros2 interface show commands to show and describe the names of the request and the response in the service named /turtlesim/get_parameters.**



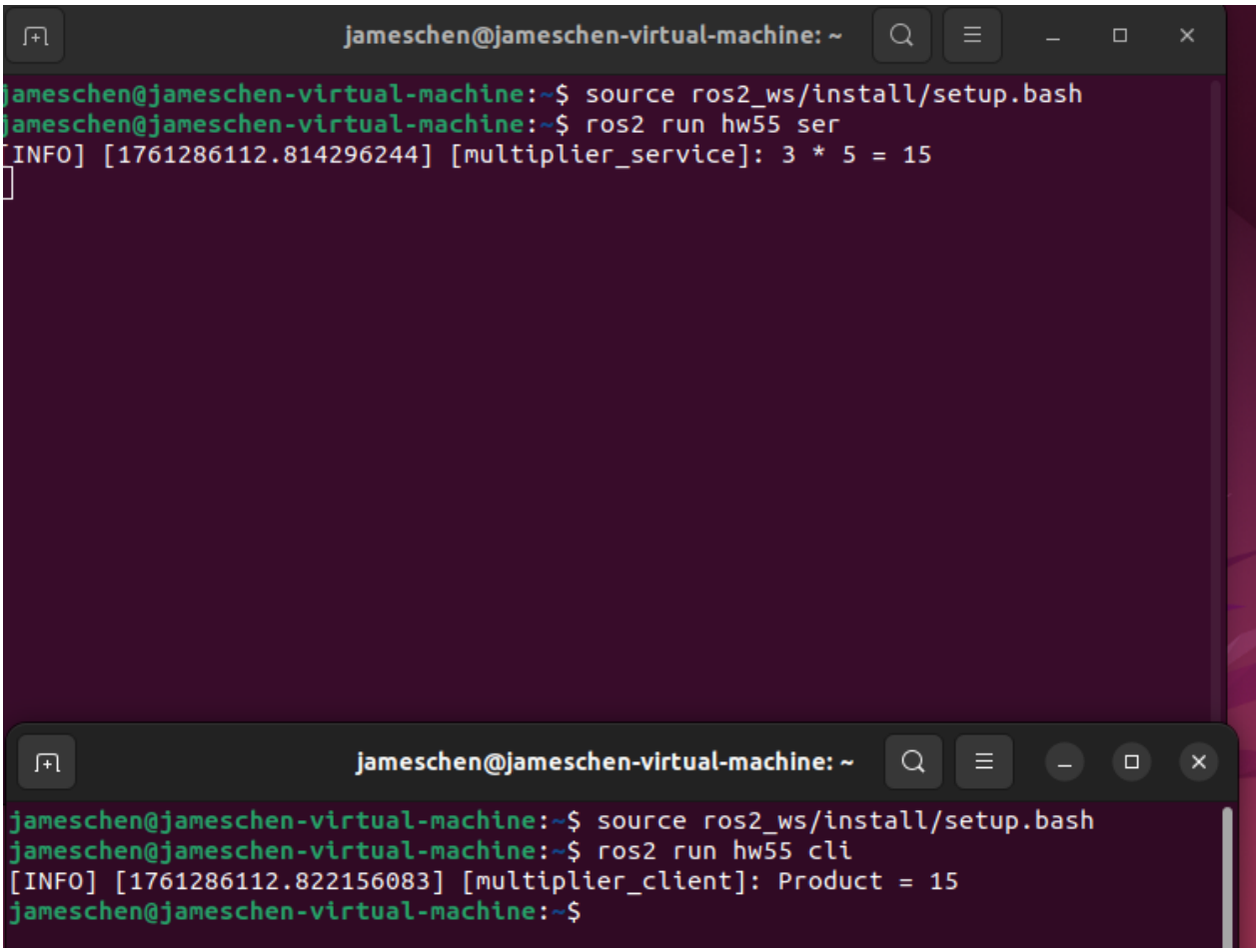

```
jameschen@jameschen-virtual-machine:~$ ros2 service list -t | grep /turtlesim/get_parameters
/turtlesim/get_parameters [rcl_interfaces/srv/GetParameters]
jameschen@jameschen-virtual-machine:~$ ros2 interface show rcl_interfaces/srv/GetParameters
# TODO(wjwwood): Decide on the rules for grouping, nodes, and parameter "names"
# in general, then link to that.
#
# For more information about parameters and naming rules, see:
# https://design.ros2.org/articles/ros_parameters.html
# https://github.com/ros2/design/pull/241

# A list of parameter names to get.
string[] names

---
# List of values which is the same length and order as the provided names. If a
# parameter was not yet set, the value will have PARAMETER_NOT_SET as the
# type.
ParameterValue[] values
        uint8 type
        bool bool_value
        int64 integer_value
        float64 double_value
        string string_value
        byte[] byte_array_value
        bool[] bool_array_value
        int64[] integer_array_value
        float64[] double_array_value
        string[] string_array_value
```

5. **In the ROS2 Humble document (https://docs.ros.org/en/humble/index.html), go through Tutorials / Beginner: Client libraries / Writing a simple service and client (Python). Describe the difference between the publisher/subscriber and the service/client in ROS2 with 100 words in Chinese or 70 words in English. Modify the example code to receive the response of the product of two numbers.**

在這兩次的作業以及官方資料裡。我總結出 Publisher/Subscriber 是單向、持續的資料發送，Publisher 把訊息丟到一個 topic，可以有任意數量的 Subscriber 去接收、訂閱，這個一對多、非同步的模式比較適合廣播系統狀態、感測器資料等等。

而 Service/Client 的模式則是 Client 送出請求後，Service 處理資料並回傳結果，這個模式是同步的，比較適合一次性指令，像是取得某項參數、做一次動作之類的請求。

完整 Code:[clickme]



6. Write a summary and give your thoughts about the first presentation. (300 words in Chinese or 200 words in English)

這一組報告的題目是”Robots, meaning, and self-determination”，我覺得他們簡報主題做得很清楚，但文字太多了，會讓我抓不到重點，加上報告者們對內容不熟悉，第一位同學就用了八分鐘，卻沒有表達更清楚，後面兩位同學也是相同的時間掌控、熟悉度問題，整場報告聽完仍不知道重點在哪裡。

我會建議將部分內容移除，只報告這篇論文作者或是報告者們自己整理的重點，並深入講解，例如，研究目的、核心方法、已解決 or 預計解決的問題，像是他們報告許多 Robots 對產業的影響，以及評斷標準(Work meaningfulness, Autonomy, Competence, Relatedness)，他們展示許多圖表對照這個判斷標準，包含年齡、性別等等，我認為可以特別強調哪一個圖表是最重要的，我有看到他們有文字是寫年齡的影響不大，但報告時這組同學很快就帶過了，整個報告沒有任何跌宕起伏，便不容易讓聽者記下重要訊息。

不過根據這組同學後續補充說明以及投影片，我學到了機器人、工業自動化與人類員工之間的關係，此篇論文指出機器人化(Robotization)不僅影響就業與薪資，更會影響人們對工作的想法、自我認同，也明確指出從事高重複性工作的人們，更容易受到機器人化影響，而使用電腦、從事社交或能獨立工作的人們則比較能維持較佳的自我認同感。這會讓我省思自己在學生時期的能力培養，增進自己的軟實力。

這次的聽者經驗讓我知道，我在讀論文時一定要找出這篇論文的重點，並做好整個報告講解的流程，完整呈現我想讓聽者知道的資訊，這樣才是有效的報告模式。